Quantifying Fairness Granularity as a Fair Ordering Policy towards MEV Mitigation for Rollups

Zeinab Alipanahloo
École de technologie supérieure
Montreal, Canada
zeinab.alipanahloo.1@ens.etsmtl.ca

Kaiwen Zhang
École de technologie supérieure
Montreal, Canada
kaiwen.zhang@etsmtl.ca

Emmanuel Awosika
Independent researcher
Georgia, USA
emmanuel@eawosika.com

2024 IEEE International Conference on Blockchain (Blockchain) | 979-8-3503-5159-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/BLOCKCHAIN62396.2024.00035

***Abstract*—Ethereum marked the beginning of stateful and Turing-Complete blockchains, where the final result of transactions depends on their execution order. This subtle distinction is of great import, especially in Decentralized Finance (DeFi) applications like exchanges or lending platforms, where execution order plays a key role in making profits or losses and gives adversarial actors enormous incentives to manipulate or influence the ordering of transactions on blockchains. Maximal Extractable Value (MEV) represents the potential profit block producers can gain by manipulating transaction inclusion within a block they create. Other blockchain participants can also extract MEV, often through tactics such as front-running attacks. The MEV problem also affects Layer-2 (L2) networks, which are a subset of stateful chains created to improve scalability for Layer-1 (L1) chains like Ethereum. Prominent examples of L2 networks include rollups such as Arbitrum and Optimism. To mitigate the MEV problem, many rollups are characterized by a single sequencer that employs the First-Come-First-Served (FCFS) transaction ordering policy, which prevents greedy reordering based on the value extracted per transaction. While FCFS policy guarantees order fairness by processing transactions according to receive times, it has some drawbacks, such as encouraging spam transactions to ensure early inclusion in a block, and sequencer orderings favoring users with lower latency. To reduce the risks of the FCFS ordering algorithm, we propose a fair ordering mechanism by adding *fairness granularity* to the original FCFS policy. We then introduce a method to measure the granularity interval of the Arbitrum chain, using a statistical technique that can be adapted for use with other L2 chains. We evaluate our proposed ordering algorithm using a dataset based on Arbitrum network specifications and quantify the accuracy of our final ordering by measuring its proximity to the *ideal ordering*. Our results show a high accuracy with different network latencies and different datasets. We also assess the effectiveness of our approach for MEV mitigation by reducing front-running compared to FCFS.**

***Index Terms*—Blockchain, Front-running, MEV, FCFS, Fair Ordering, Latency Optimization**

I. Introduction

Blockchains have historically achieved data integrity and immutability through a network of nodes reaching consensus on the validity of transactions recorded in the distributed ledger. Fairness in transaction ordering within blockchain technologies occurs when the final block of transactions is established without favoring any particular participant, such as miners or validators. Consensus protocols adopted in many blockchains do not enforce rules on the ordering of transactions produced by block producers. Miners or validators can exploit their capability to preview the network's upcoming state and manipulate transactions by reordering, including, or excluding them to generate profits. This profit-seeking behavior is known as Maximal Extractable Value (MEV) extraction can lead to significant issues for users, applications, and overall network robustness. This becomes significant with the rapid expansion of Decentralized Finance (DeFi) projects, leading to a notable increase in MEV extraction within the Ethereum network [1].

One prominent approach to obtain MEV is through a front-running attack, which involves submitting a transaction with higher transaction fees to surpass a pending transaction in the execution queue [2]. Front-running attacks form a dominant method to extract MEV, as they enable individuals to leverage the transaction prioritization mechanism for their own financial gain. It is important to note that transaction ordering manipulation also occurs on off-chain or layer-2 (L2) networks, where single or multiple sequencer nodes are responsible for determining the final transaction order. L2 solutions are designed to improve the scalability and performance of blockchain networks by processing transactions off-chain [3]–[5]. Rollups are a specific type of L2 solutions with a single sequencer. The sequencer, under the control of the rollup operator, is a central authority responsible for the ordering and execution of transactions. Rollup chains employ a single sequencer to ensure deterministic transaction ordering, simplify system architecture, and enhance efficiency and security, although it may have certain drawbacks. Currently, all rollups (e.g., Arbitrum/Optimism) use some forms of the First-Come-First-Served (FCFS) ordering policy that guarantees transactions are ordered as they received [6]. The traditional FCFS approach offers certain benefits; for example, it is straightforward to implement and provides a degree of protection against transaction reordering attacks. However, it results in an elevated volume of spam transactions aiming for early inclusion in a block, potentially leading to a waste of block space. In addition, network latency, influenced by geographical factors, is a crucial concern for fairness in FCFS chains, causing transactions from users with high latency to be queued much later than those with lower latency. MEV searchers may choose to co-locate with sequencer nodes to

This research is supported by Flashbots with grant No. FRP-25.

2834-9946/24/$31.00 ©2024 IEEE
DOI 10.1109/Blockchain62396.2024.00035

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on February 01,2026 at 15:00:47 UTC from IEEE Xplore. Restrictions apply.

achieve the most favorable transaction ordering. The pursuit of Latency Optimization (LO) for front-running by MEV profiteers is commonly referred to as *latency wars* [1], [7].

The paper focuses on enhancing the fairness of transaction ordering in rollup protocols with a single sequencer. The study utilizes Arbitrum as a representative case to implement and evaluate the proposed improvements. The main contributions of this paper are summarized as follows:

- We design an algorithm based on the concept of *fairness granularity* to enhance transaction ordering fairness, addressing issues associated with the FCFS ordering.
- We calculate the *fairness granularity* interval using a statistical method applied to Arbitrum block time dataset as a case study, and this approach can be extended to other Rollup chains.

The rest of this paper is organized as follows: In Section II, we explain background concepts related to the topic. Section III explains some previous studies on the MEV problem. The proposed fair ordering algorithm is in Section IV, while Section V presents our method for calculating the granularity interval. Section VI provides the experiments followed by simulations and evaluations. Finally, Section VII concludes our paper.

II. Background

In this section, we delve into foundational concepts relevant to the subject, including front-running attacks and L2 scaling solutions.

A. Front-running

In its formal definition, MEV represents the highest value or profit a miner can attain from pending or confirmed transactions within a network. Transactions are stored in a public pool called mempool, allowing anyone to monitor and identify profit making opportunities. Miners, validators, sequencers or block proposers hold the authority to include, exclude, or censor transactions within a block, allowing them to exploit this power to maximize their revenue at the expense of normal users. Other participants in the network, known as MEV searchers or even bots, can also extract MEV. Miners are trying to give higher priority to transactions which offer higher transaction fees to gain more profit. This rational behavior of miners can be exploited by front-running attacks which is the most common method of extracting MEV. In this attack, a transaction is placed ahead of another in the execution queue by bidding higher transaction fees. Malicious actors exploit this by executing transactions with higher gas fees to front-run and extract profit before the original transaction is processed. When multiple front-runners compete, it leads to a Priority Gas Auction (PGA), where the highest gas fee wins (Fig. 1). However, this can cause network congestion and result in higher fees for regular users not participating in the auction [1]. In certain situations, a miner or block proposer might choose to censor or delay a specific transaction. This could be to prioritize their own transaction in order to profit, particularly

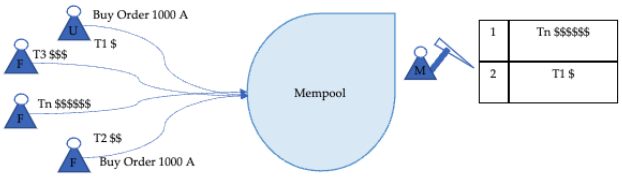


Fig. 1: Priority Gas Auction Example on Ethereum Network

TABLE I: Prominent Rollup protocols with their sequencing model

Protocol	Rollup Type	Sequencing	Ordering Policy
Arbitrum	Optimistic	Single sequencer	FCFS
Optimism	Optimistic	Single sequencer	FCFS
Starknet	Zero-knowledge	Single sequencer	FCFS
Zksync	Zero-knowledge	Single sequencer	FCFS
Metis	Optimistic	Decentralized sequencers	FCFS

if the potential profit exceeds the combined block reward and transaction fees.

B. L2 Scaling Solutions

L2 solutions offer a way to scale the underlying chain or Layer-1 (L1) chain. Each L2 protocol has its own execution environment, resembling the Ethereum virtual machine. The key concept involves deploying DApps (decentralized applications) on L2 protocols, enabling computations and storage to take place off-chain. Rollups are a type of the L2 solution, in which computations and the application's state are kept in an off-chain protocol, and the proof data of transactions are stored on L1. In this study, we focus on the rollup protocols as an efficient and general-purpose off-chain solution. Rollups aggregate multiple transactions into a single batch and perform most computation off-chain, using the underlying network for security. There are two main types of rollups: optimistic rollups assume correctness unless disproven. In contrast, Zero-Knowledge rollups use cryptographic proofs to validate transactions. A crucial component in the architecture of rollups is the sequencer. The sequencer plays a vital role in maintaining the correct order of transactions and ensuring the consistency of the network state. As is shown in Table I, most of the Rollups have a single sequencer. The users send their transactions to the sequencer of the Rollup chain. The sequencer then collects these transactions and orders them in a queue based on their received time, creating a batch of transactions. Periodically, the sequencer commits the batch to the underlying network, thereby ensuring the integrity of the transactions and the rollup's state. Transaction ordering and execution can be separated in rollup protocols, but in practice, they can be gathered in a single node [6], [8].

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on February 01,2026 at 15:00:47 UTC from IEEE Xplore. Restrictions apply.

III. Related Works

Efforts to minimize MEV exploitation include limiting block builders' ability to reorder transactions arbitrarily through applying fair ordering policies on sequencer nodes. Fair ordering policies can be applied on a single sequencer or decentralized sequencing settings.

The authors in [9] proposed a novel fair ordering algorithm known as **TimeBoost** for a single sequencer that leverages an auction mechanism alongside time intervals. Transactions can expedite their arrival times within a given interval to the sequencer node, thereby increasing their priority through bidding. **Metis** is a decentralized sequencing protocol recently deployed. Every time a sequencer is selected to propose an ordering of transactions. However, on each round the sequencer acts similar to a single sequencing setting and orders transactions based on it's local mempool [10]. Kelkar et al. [11] introduced **Themis**, a permissioned and leader-based sequencing protocol, with fair ordering capabilities. Every sequencer orders their transaction based on FCFS algorithm. In each cycle, one of the sequencers selected as the leader. The leader algorithm to create a final order based on the local orderings of transactions from other sequencers is called *Batch-Order-Fairness* [11]. Additionally, various distributed fair ordering strategies exist that prioritize transactions tx over tx' contingent upon all honest nodes receiving tx before any honest node receives tx'. A limitation of these approaches is their reliance on synchronized clocks [12], [13]. Mamageishvili et al. another initiative introduced an **age-aware** fairness mechanism to sequence transactions [14], while further research has explored reputation systems to foster integrity among block proposers, and employing random selection for fair transaction ordering [15].

A distinct strategy for reducing MEV involves enhancing transaction privacy. This approach typically conceals the details of transactions during the ordering phase, with decryption occurring only after their sequence is established, thereby discouraging miners and others from engaging in front-running or similar manipulative tactics. Several methods within this category adopt **threshold decryption**, necessitating a committee of nodes for transaction decryption [16]–[18]. Despite its potential to reduce front-running and censorship, this methodology faces challenges such as the misbehaving of the committee and communication complexity during the decryption phase.

In this study, we propose a fair ordering policy for a single sequencer to address challenges associated with the classic FCFS algorithm, including latency wars and spamming.

IV. Fair Ordering Algorithm for FCFS Chains

In this section, we describe our proposed fair ordering algorithm, designed to improve the original FCFS ordering policy used on Rollup chains. The general idea is to move away from a strict transaction queue order, which may not be equitable, and adopt a modified version of the FCFS ordering algorithm.

Fig. 2: Applying Fairness Granularity g on a Timeline

To define the algorithm we leveraged the concept of *fairness granularity* which is defined in Definition IV.1 [11].

Definition IV.1 (Fairness granularity)
For granularity g, we consider that timestamps are bucketed into slots of interval g time each (e.g., $[0; g)$, $[g; 2g)$ and so on). Events within a time bucket are assumed to happen at the same time.

The *fairness granularity* serves as a fundamental building block for the algorithm. It allows for the categorization of timestamps into discrete time intervals or slots with granularity g as is shown on Fig. 2, and treats events within each interval as occurring simultaneously. By utilizing this definition, the fair ordering algorithm aims to create a more equitable mechanism for transaction ordering. In practical terms, the fair ordering algorithm considers the temporal aspect of transactions, ensuring that their ordering is not solely based on their receiving time. According to the *fairness granularity* definition, the steps for our proposed fair ordering method is shown in Algorithm 1.

Algorithm 1 Transaction Ordering Algorithm

Initial Ordering:
The sequencer orders transactions based on their received times (same as classic FCFS).
Fairness Granularity:
It divides the timeline into the intervals with the calculated granularity g.
Ordering within an interval:

- It is assumed that the transactions are received at the same time within an interval, but the order of the transactions across these intervals is based on the first step.
- In every interval, transactions are selected using a specific mechanism to determine the priority between them.

Selection Mechanism:

- **Random Selection:** The transaction within a batch or interval can be chosen randomly.
- **Batch Auction:** Users can bid a certain amount of tokens or fees to increase the likelihood of their transactions being processed early.

A. Discussion on the Algorithm Specifications

Opportunities for extracting MEV frequently arise within the network, offering significant profit potential through activities such as trading specific token pairs on DEX (decentralized

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on February 01,2026 at 15:00:47 UTC from IEEE Xplore. Restrictions apply.

exchange) platforms. Consequently, MEV transactions rapidly saturate the network, often within milliseconds, to capitalize on these lucrative prospects. However, employing a suggested sequencing algorithm within the context of a single sequencer with a FCFS ordering policy can help mitigate front-running through the following means:

- By giving an equal chance for every transaction within an interval g, it can reduce the impact of network latency on front-running. Specifically, the fact that a user has the lowest network latency does not mean that their transaction will be processed sooner than others, while this problem exists in the original FCFS. Offering equal opportunities to every transaction within an interval g serves to mitigate the impact of network latency on front-running. It means, that even if a user possesses the lowest latency, it does not guarantee their transaction will be prioritized ahead of others. This stands in contrast to the original FCFS policy, where investment in LO often correlates with more optimal positioning in the sequence.
- The algorithm discourages MEV searchers from flooding the network with spam transactions to secure early inclusion. Instead, it encourages a more fair and balanced transaction ordering procedure.

Selection mechanism: It is mentioned in Step 4 of Algorithm 1 which can be an **batch auction** mechanism or a **random selection** or any other customized mechanism. Using randomness as the selection mechanism ensures a fair and unbiased process to select transactions. It provides equal chances for all transactions to be included in an interval. This approach prevents any single party from having control over the ordering of transactions. While random selection can be a fair and unbiased approach to determine the priority of transactions within an interval, it does come with some trade-offs. The concern lies in the possibility of malicious users abusing randomness by flooding the network with many transactions. This flood aims to improve their chances of getting their transactions prioritized and processed swiftly.

A **batch auction selection** mechanism allows users to bid for priority in having their transactions included within a batch or time interval. Users can offer tokens or fees to boost the chances of their transactions being processed sooner. This introduces a competitive aspect, enabling users to pay more to prioritize their transactions. The key aspect of this selection mechanism is its avoidance of the congestion issues present in PGA. Unlike PGA, where all network transactions are involved in the auction, here only a limited number of transactions within an interval participate, preventing network congestion. The only work that is done about this kind of selection method is TimeBoost [9] where transactions bid to buy time and advance in the queue, limited to a specific time interval g.

Near optimal granularity determination: It is the most critical aspect of the algorithm, as highlighted in the Themis work [11]. Setting the granularity too small makes it similar to a classic FCFS approach. The authors considered using the minimum possible value of g ($\epsilon \approx 0$) [11]. Conversely, a very large granularity leads to an increased number of transactions falling into each interval, resulting in a time-consuming process for ordering within the granularity intervals. Striking a balance between smaller and larger granularity involves specifying the size of time intervals for ordering transactions in FCFS blockchains, where events within each interval are assumed to occur simultaneously.

Suitability of the calculated granularity: The suitability varies across different blockchains due to their unique characteristics. A crucial factor in determining the appropriate granularity is the **block time**, which can differ significantly among various blockchain networks. Factors such as block size, network congestion, and the consensus algorithm employed (e.g., Proof-of-Work, Proof-of-Stake) influence the block time. Block time plays a vital role in blockchain design, impacting transaction throughput and confirmation speed. Shorter block times result in faster transaction confirmation, while longer block times may enhance security at the expense of slower confirmations. Transaction volume is another factor that is affected by the block time. Higher transaction volumes might require smaller granularity to avoid congestion within a single slot.

V. Granularity Calculation Method

The proposed granularity calculation method is applicable to any Rollup network. In this research, the granularity computation focuses on the Arbitrum network, chosen for its prominence as an L2 scaling chain. Arbitrum employs a single sequencer node and adopts the FCFS ordering policy [6]. We present our method to compute the granularity for the Arbitrum chain. Initially, we introduce the *burst period* as a standard to estimate the granularity interval. Subsequently, we dig into the process of calculating the *burst Period* on the Arbitrum network.

A. Burst Period as a Fairness Granularity Indicator

A burst period has been identified as a suitable indicator for computing the granularity interval. The *burst period* definition is based on the work of Chaboud et al. [19], who explored price discovery in electronic markets. It quantifies as the average time between two trade events (such as trades or changes at the top of the order book), and is relevant to traditional markets where each trade prompts a new price for a currency or a stock [19]. Drawing from this concept, we applied it to our problem of MEV transactions, which primarily occur within DeFi projects. In MEV transactions, participants seek to gain higher priority over other users or MEV searchers, particularly on DEX platforms like Uniswap. This implies that the most intense competition among users of a DApp occurs between two consecutive trades. Since every request sent to DApps requires transaction execution on the network, we defined the burst period in the context of Blockchain Definition V.1.

Definition V.1 (Burst period)
The average duration between the reception times of two consecutive transactions at the sequencer node. If there are n

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on February 01,2026 at 15:00:47 UTC from IEEE Xplore. Restrictions apply.

transactions queued up, the burst period can be calculated as the average time between the reception times of the transaction pairs (tx_i, tx_{i-1}) *in ms(millisecond).*

$$\frac{\sum_{i=1}^{n-1}(receive(tx_i) - receive(tx_{i-1}))}{n-1} = ms \qquad (1)$$

In this study, we utilize the **burst period** as a metric to measure the *fairness granularity*, enabling us to bucket pending transactions in a queue into intervals corresponding to the burst period's duration.

B. Limitations of the Burst Period Computation

To accurately estimate the burst period on Arbitrum, it is crucial to access transaction data on the platform. Publicly available data includes recorded transactions and their **block timestamps** on Arbitrum, which are given in Unix timestamp format but may lack the necessary precision for detailed analysis. Transactions are sent directly to the sequencer node when interacting with DApps on Arbitrum. The sequencer node is operated by the Arbitrum company, limiting the ability to monitor pending transactions and pinpoint exact transaction receipt times. To overcome these limitations, we created our own dataset of transactions with their specific receipt times using a statistical method based on the network parameters of Arbitrum, as detailed in Section VI.

VI. Experiments

The fair ordering policy algorithm, described in Section IV aims to enhance fairness on classic FCFS chains. To address the challenges of calculating the granularity, we introduced the *burst period* as a suitable indicator for the *fairness granularity*. In the subsequent steps, we computed the *burst period*. Initially, we generated a synthetic dataset to determine the *burst period*. After that, we employed our proposed algorithm with the derived granularity interval on a newly generated synthetic dataset. Ultimately, we evaluated the final sequence of transactions.

A. Realistic Data Generation Method

To generate a synthetic dataset of time intervals between transactions, we used the Weibull distribution in (2) [20]. A Weibull distribution is a continuous probability distribution that is commonly used for modeling the time between two consecutive events and is a powerful tool that can help us to better analyze and model our data.

$$f(x; \lambda, k) = \frac{k}{\lambda}(\frac{x}{\lambda})^{k-1}e^{-(x/\lambda)^k} \qquad (2)$$

The Weibull distribution is flexible and can take on different forms depending on the values of its two parameters: the shape parameter (k) and the scale parameter (λ). The shape parameter controls the shape of the distribution curve, while the scale parameter determines the location of the curve along the x-axis. The scale parameter for the Weibull distribution is typically determined based on the characteristics of the system or process being modeled. Seeing as we want to model the average interval between the occurrence of two transactions

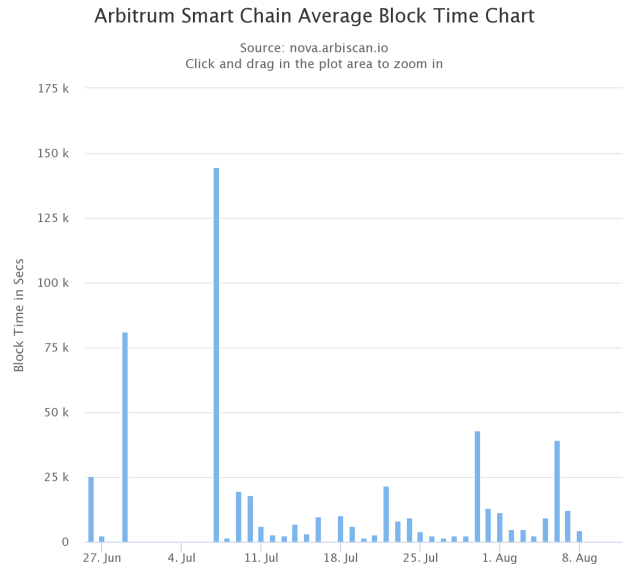


Fig. 3: Arbitrum Nova ABT (in seconds) for each day

on the blockchain network, we can use block time to estimate the scale and shape parameters. Note that in Section IV-A, the block time represents how long it takes to create a new block on the blockchain (measured in seconds); it is also a good indicator of transaction latency (the time it takes the network to process a transaction). The goal is to estimate the *burst period* for the Arbitrum network using Weibull distribution. If we can estimate the distribution of average block times in Arbitrum, we can determine the Weibull parameters and generate periods between events based on that. Eventually, we can compute the burst period.

*1) **The Block Time Data Extraction**:* We extracted the Ethereum, Arbitrum One, and Arbitrum Nova Average Block Time (ABT) datasets from Etherscan and Arbiscan block explorers, respectively (using the CSV-format export option). Each record of the dataset represents the ABT in one day. As a result, the data size is the number of days included in the dataset. We collected the Arbitrum chains block time data spanning from the beginning of the chains up to August 2023. The Ethereum extracted data is related to the chain's start until September 2022 (during Ethereum's utilization of the proof-of-work consensus algorithm).

Fig. 3 shows the ABT data for Arbitrum Nova. You notice some spikes in the chart. These are the uncommon cases with recorded exceptionally long ABTs that may be due to network outages or technical problems happening in the network. Consequently, these cases are eliminated from the dataset.

Arbitrum One: We derived two ABT datasets for Arbitrum One from the CSV file provided by Arbiscan. The initial ABT dataset is known as Arb-One-1. According to the information provided in Table II, the maximum ABT in the Arb-One-1 dataset is 36.5 seconds (s), while the median and mean ABT

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on February 01,2026 at 15:00:47 UTC from IEEE Xplore. Restrictions apply.

TABLE II: ABT Datasets Specifications

ABT Datasets	Max (s)	Median (s)	Mean (s)	Min (s)	Data Size (No. of Days)
Arb-One-1	36.5	0.85	2.27	0.24	765
Arb-One-2	2.03	0.39	0.64	0.24	577
Arb-Nova-1	31.5	4.53	4.77	0.36	379
Arb-Nova-2	4.76	0.95	1.46	0.36	198
Ethereum-POW	30.31	13.85	14.51	4.46	2604

values for the entire 765-day period are much less than this time. As a result, we formed a new dataset called Arb-One-2 by removing outliers, which incorporates ABTs below the mean value, representing approximately 77% of the original Arb-One-1 dataset.

Arbitrum Nova: Nova is similar to the classic Arbitrum chain but with an added trust assumption in the shape of a data availability committee. The Nova blockchain has a shorter lifespan, resulting in a smaller amount of available data compared to the Arbitrum One network. Similar to Arbitrum One, the original dataset for Arbitrum Nova (Arb-Nova-1) contains ABTs with a maximum of $31.5s$ (as shown in Table II). However, the mean and median ABT on Arbitrum Nova are around $5s$. Just like in the previous dataset, we created a new dataset (Arb-Nova-2) by selecting ABTs less than the mean value, which here is $4.77s$.

Ethereum: Ethereum stands as a robust blockchain with a substantial operational track record and a multitude of nodes, rendering it well-suited for burst period computation. However, the Ethereum-POW ABT data pertains to the proof-of-work consensus mechanism (before September 15, 2022). Ethereum's block time, as reported by diverse sources, is around 14 seconds. Our computations likewise reveal an average ABT of 14.51.

*2) **Weibull Parameters Estimation**:* In our analysis, we employed the Python module *scipy.stats* [21] to determine the parameters *shape* and *scale* of the Weibull distribution using the ABT dataset created in Section VI-A1. The $scipy.stats.weibull_min.fit$ function in SciPy fits a Weibull distribution to the provided dataset using the method of **Maximum Likelihood Estimation** (MLE). MLE is a statistical method used to estimate the parameters of a probability distribution that best describes the data. For the Weibull distribution, the parameters to be estimated are the shape and scale parameters. The method uses MLE to find the values of these parameters that maximize the likelihood of the data coming from a Weibull distribution. In other words, it finds the parameters that make the data most probable under the assumed Weibull distribution. The results are shown in Table III for each extracted ABT dataset.

3) Burst Period Calculation: Based on the Weibull parameters calculated in Table III from the five extracted ABT data points using the $stats.weibull_min.rvs$ function, we proceeded to generate a synthetic dataset comprising **one million** records of transactions. These transactions were generated with time intervals between consecutive transactions following the Weibull distribution. This process was repeated *50 times*. In each iteration, we computed the burst period based on (1) of the generated time intervals and obtained the average of these 50 burst periods as the final *burst period*, presented in Table IV.

TABLE III: Estimated Weibull Parameters for Different Datasets

Average Block Time (ABT) Datasets	Scale	Shape
Arb-One-1	3.7870	0.4692
Arb-One-2	0.3331	0.6864
Arb-Nova-1	4.2092	0.8956
Arb-Nova-2	1.1446	1.0827
Ethereum-POW	1.2514	0.8509

TABLE IV: Burst Periods for Synthetic Transaction Data

	Arb-One-1	Arb-One-2	Arb-Nova-1	Arb-Nova-2	Ethereum-POW
Burst Period (ms)	8576	429	4440	1110	1360

B. Burst Period Results Analysis

Based on the Weibull parameters calculated in Table IV final burst periods were computed for the five datasets. In particular, the new burst period estimated for the Arbitrum One chain, derived from the Arb-One-2 dataset, is $429ms$, which is lower than the Arb-One-1 dataset with outliers. The variance between the two Arbitrum datasets suggests that the *burst period* is not fixed and depends mainly on the block time. However, it is reasonable to believe that burst periods on the Arbitrum network will be around $429ms$ most of the time.

Regarding the Arbitrum Nova chain, when analyzing the *burst period* for Arb-Nova-1, the *burst period* value is estimated to be roughly $4s$. However, to enhance the accuracy and reliability of this measurement, the process involves eliminating outliers from the dataset. The refined *burst period* result reveals a reduced value of approximately one second. This suggests that there exists a shorter period during which a significantly higher rate of transactions or events occurs.

Ethereum has a public mempool without any fair ordering policy, so we cannot apply our optimized FCFS policy on Ethereum. We computed Ethereum's burst period as the underlying blockchain for the rollups. Our calculation shows $1360ms$ for the Ethereum-POW dataset. Ethereum block time is around 14 seconds, which is the time it takes to mine a new block and add it to the blockchain. However, the average time between two transactions being processed can be much less because multiple transactions are included in each block. This value indicates the duration within which there is a notable surge in transaction or event activity on the Ethereum network, followed by subsequent periods of comparatively lower activity.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on February 01,2026 at 15:00:47 UTC from IEEE Xplore. Restrictions apply.

C. Simulation

We conducted a simulation using generated transaction data. After establishing the granularity interval through statistical analysis, we proceeded to implement the algorithm on a dataset of transactions. The algorithm assigns priorities to transactions within each interval, ensuring fairness and reducing the impact of network latency on transaction ordering. The final sequence of transactions, as determined by the sequencer using the proposed fair ordering algorithm, is saved for the evaluation phase. A dataset consisting of 1000 transactions based on the calculated Weibull parameters is generated for the Arbitrum network. In the following, we have introduced the parameters of the generated dataset for the evaluation phase.

- **Send:** Represents the time when a transaction is generated or sent to the sequencer at the client side.
- **Latency:** The time it takes for a transaction to reach the sequencer node.
- **Receive:** indicates the time at which each transaction is received by the sequencer node, calculated based on $(Send + Latency)$.

Utilizing the determined granularity interval, as indicated in Table IV, we implemented it on the simulated dataset. Within each granularity interval, we employed a random selection mechanism, chosen for simplicity, to prioritize the transactions within an interval g and to create the final ordering.

a) Transaction Latency: The computation of transaction latencies involves employing the Weibull distribution alongside parameters computed based on the block time data of each blockchain. Utilizing real network data to extract these Weibull parameters, the resulting latency values precisely emulate real-world network behaviors. Fig. 4 shows bar charts for each blockchain regarding the generated latencies. Every bar in the charts represents the count of latencies that are below a specific threshold. As it shows, most of the latencies are less than one second, but we have mixed values. The data scattering indicates the presence of users across diverse geographical locations and varying network capabilities. The latency values produced across various datasets contain not only low and high latencies but also encompass multiple latency profiles. However, as illustrated in Fig. 4, latencies associated with Arb-One-1 and Arb-Nova-1, including outliers, indicate a broader range.

b) Selection Mechanism: It is important to highlight the selection mechanism. Opting for random selection might not provide a viable solution, as it can introduce issues, such as the persistence of spam problems. Even with random selection, the risk of encountering spam remains. Attackers could exploit this by bombarding the system with numerous transactions, thereby elevating their likelihood of being randomly chosen within a specific time frame. Instead, adopting an auctioning approach could offer a more effective selection mechanism for each interval.

D. Evaluation

To quantitatively estimate the fairness of the proposed method and to be able to compare it with the Themis work [11], we adopted a similar evaluation methodology as outlined in [11]. Our evaluation focuses on measuring how closely our transaction ordering aligns with the *ideal ordering*. This ideal setting involves ordering transactions based on their generation time at the client side, rather than when they are received on the sequencer side. The evaluation procedure involves comparing the final sequence of transactions with their original order at the time of generation for each transaction pair. This allows us to quantify the degree of deviation from the *ideal ordering* and assess the fairness and accuracy of our proposed method in maintaining the chronological order of transactions. Following the procedure outlined in the Simulation section, once 1000 transactions were generated, we proceed to determine the number of transaction pairs $(tx; tx0)$ such that tx was transmitted prior to $tx0$ ($Send(tx) < Send(tx0)$). Additionally, we verify that tx precedes $tx0$ in the final output produced by the proposed algorithm.

TABLE V: The results of the evaluation Stage, including the precision of the suggested algorithm concerning the ideal ordering and the amount of LO side effects reduction in comparison to FCFS

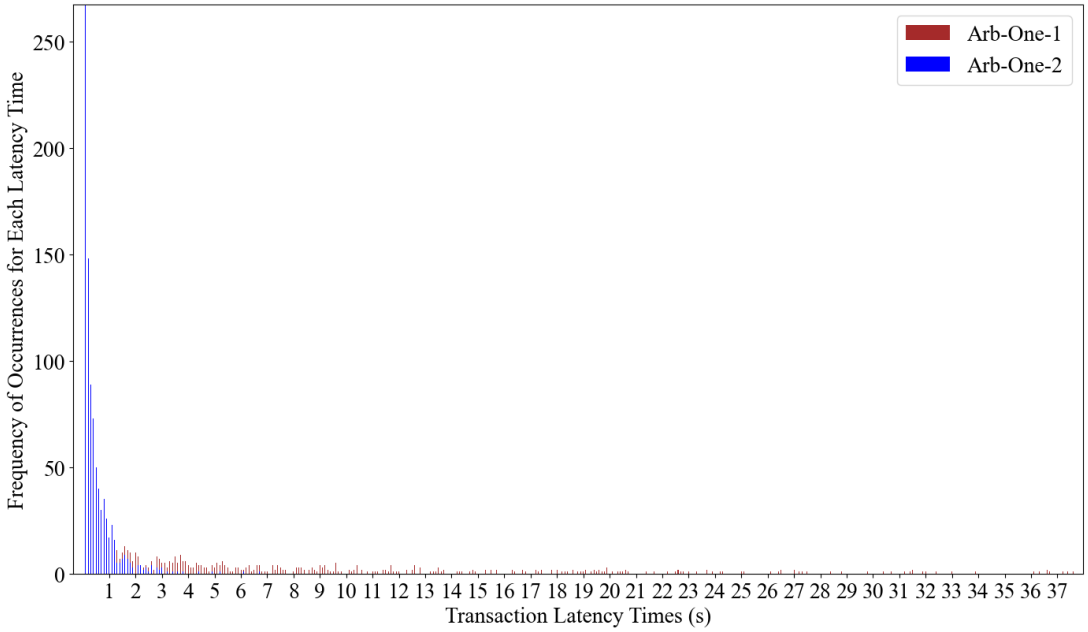
	Arb-One-1	Arb-One-2	Arb-Nova-1	Arb-Nova-2
According to the Ideal Ordering (%)	69.70	73.68	74.22	73.92
LO Effect Reduction (%)	37.97	36.45	34.95	32.92

Themis protocol employs an exponential distribution for generating receive times, characterized by a mean r, which serves as the network ratio parameter governing network latency. When r is exceedingly low ($r \ll 1$), Themis achieves **100%** accuracy, indicating that in the absence of latency, transactions are received in the ideal order. However, by increasing latency, the count of transaction pairs conforming to the ideal order diminishes. In the best-case scenario, the Themis protocol attains **60%** accuracy in a network with maximum latency. Further details of these experiments can be found in [11].

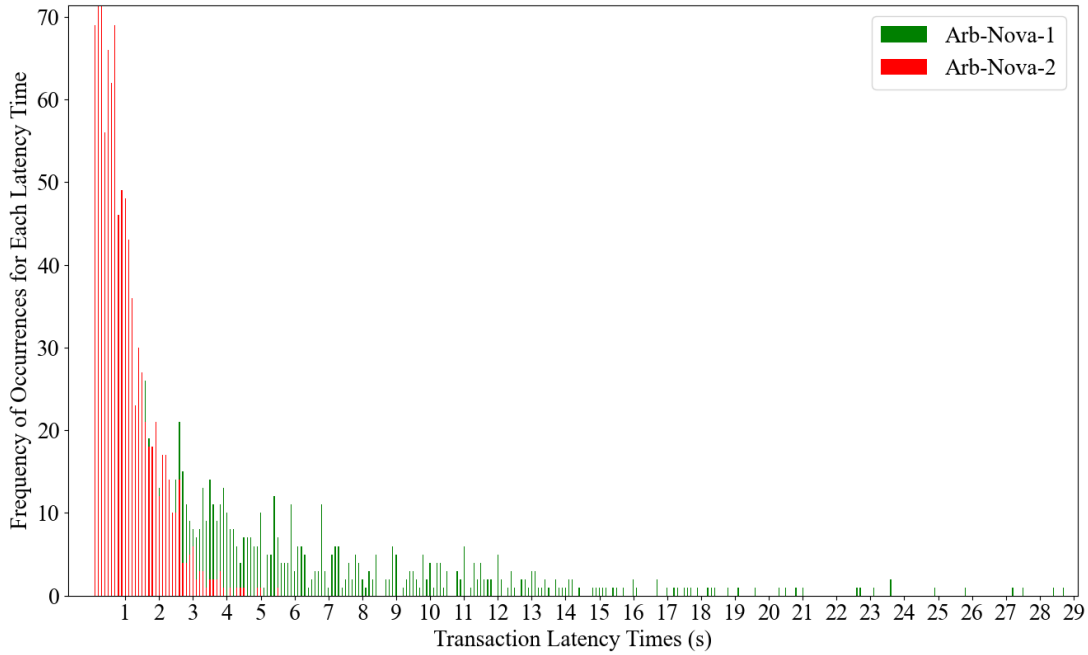
Table V displays the computed accuracies of our proposed method. In this regard, the concept of *ideal ordering* serves as a metric for quantifying our assessment of the proposed ordering algorithm. Through various transaction datasets generated using diverse Weibull parameters, we achieved accuracies of approximately **70%**. This suggests that over **70%** of transaction pairs are organized based on their generation time, meaning they align with the *ideal order*, even in scenarios involving transactions with varying latencies (see Section VI-C0a). Regarding the latency times associated with the Arb-One-1 and Arb-Nova-1 datasets, which exhibit greater diversity and scattering, we achieved a comparable level of accuracy compared to scenarios with lower latencies.

a) ***Evaluation of the algorithm regarding the LO problem:*** In the previous step, we assessed our suggested fair ordering policy regarding its proximity to the ideal ordering, without taking into account transaction latency. We conducted an experiment to evaluate how much our approach can alleviate the impact of aggressive LO, thereby diminishing the

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on February 01,2026 at 15:00:47 UTC from IEEE Xplore. Restrictions apply.



(a) Arbitrum One Chain



(b) Arbitrum Nova Chain

Fig. 4: Simulated transaction latency times using the Weibull distribution

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on February 01,2026 at 15:00:47 UTC from IEEE Xplore. Restrictions apply.

occurrence of front-running.

The goal is to count the transaction pairs, the following holds for them: For a pair $(tx, tx0)$ where $Send(tx) < Send(tx0)$ and $FairOrdering(tx) < FairOrdering(tx0)$ then $Latency(tx) \geq Latency(tx0)$. This means that although tx has a higher latency than $tx0$, it should be given higher priority than $tx0$ in the final fair ordering because tx was sent earlier by the user, unlike FCFS which only considers the receive time of transactions which provides an incentive to optimize latency for front-running. The amount of improvement is computed based on the ratio of the number of transaction pairs matching the above definition to the total number of fairly ordered pairs according to the ideal ordering (previous step). The percentage of improvement with respect to FCFS regarding each dataset is depicted in Table V. The results reveal a notable advancement of over **30%** in all cases compared to FCFS, specifically in reducing LO and consequently mitigating front running.

E. Fair Ordering in Decentralized Settings

That said, rollups rely on a single sequencer node, assuming their honest behavior under a fair ordering policy. Nonetheless, this introduces a trust assumption. A malicious sequencer has the potential to front-run transactions or collude with MEV searchers for transaction reordering manipulation. To mitigate the risk and ensure network reliability, a solution involves transitioning to decentralized sequencing.

Our proposed algorithm is also adaptable to decentralized protocols such as Themis [11], where individual sequencers can utilize it for local ordering. Themis [11] stands as a prominent leader-centered consensus protocol for achieving equitable ordering, characterized by a communication cost of $O(n)$ among sequencer nodes. In the Themis work [11], individual users transmit their transactions to all sequencer nodes within the network. Subsequently, each sequencer orders the transactions following their received time at the sequencer node, employing the FCFS principle. Following this, the sequencers send their local orderings to the leader node, which ultimately assembles a conclusive sequence of transactions by integrating all the local orderings. We note the possibility of Condorcet cycles [22] appearing when a leader node attempts to aggregate ordering preferences of individual nodes into a final sequence (due to differences in message receipt times across protocol nodes; see [11] for examples and discussion).

We suggest employing the calculated *fairness granularity* appropriate to the particular blockchain as an alternative to the traditional FCFS approach for the local ordering of sequencer nodes. In this approach, each sequencer applies the determined granularity to the FCFS ordering and forwards transaction intervals containing simultaneous transactions to the leader. This departs from a fixed transaction order, allowing for more fairness and flexibility in handling simultaneous transactions. We should also note that the fairness granularity interval employed by our algorithm is established according to the local time at each node which eliminates the need for synchronized clocks among sequencer nodes to maintain accuracy in the ordering process, while leaving room for the fairness granularity parameter to change without any impact to the protocol. Eventually, the adaptation of the leader algorithm should be considered based on receiving batches of transactions.

VII. Conclusion

In conclusion, our research has provided valuable insights into estimating the burst period and determining the appropriate fairness granularity interval for fair transaction ordering. By analyzing the distribution of ABTs and utilizing the Weibull distribution, we successfully computed the *burst period*, representing the average time between transactions. It is essential to recognize that granularity intervals may vary significantly across different blockchains, influencing their overall performance and usability. Although the results are related to Arbitrum, the fair ordering algorithm can also be applied to other Rollup chains. Granularity is determined by analyzing past blockchain data and estimating the granularity parameter, which can then be easily applied to a sequencer node. We employed this approach on rollups utilizing a single sequencer and FCFS as their ordering policy. However, it can be extended to distributed protocols and holds the potential to become a distinctive area for future research.

In order to assess the fairness of the algorithm, we employed the concept of *ideal ordering*. By applying the proposed method with the calculated granularity, we achieved a satisfactory level of accuracy across various network latency scenarios. We have also established a methodology to gauge the enhancement of our protocol in minimizing the impact of aggressive LOs on front-running, as compared to the traditional FCFS, and observed a notable improvement.

Looking ahead, future work will focus on implementing the proposed fair ordering mechanism in a distributed setting and integrating it with privacy preserving approaches to further enhance security and fairness. By continuously exploring and refining these strategies, we aim to contribute to the ongoing efforts to mitigate MEV extraction and promote a more equitable blockchain ecosystem.

References

[1] P. Daian, S. Goldfeder, T. Kell, *et al.*, "Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability," in *2020 IEEE Symposium on Security and Privacy (SP)*, IEEE, 2020, pp. 910–927.

[2] S. Eskandari, S. Moosavi, and J. Clark, "Sok: Transparent dishonesty: Front-running attacks on blockchain," in *International Conference on Financial Cryptography and Data Security*, vol. 11599, 2019, pp. 170–189.

[3] G. Konstantopoulos, *(almost) everything you need to know about optimistic rollup*, Paradigm Research, 2021. [Online]. Available: https://www.paradigm.xyz/2021/01/almost-everything-you-need-to-know-about-optimistic-rollup.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on February 01,2026 at 15:00:47 UTC from IEEE Xplore. Restrictions apply.

[4] T. Schaffner and F. Schaer, "Scaling public blockchains—a comprehensive analysis of optimistic and zero-knowledge rollups," *A comprehensive analysis of optimistic and zero-knowledge roll-ups, University of Basel*, 2021.

[5] A. Gangwal, H. R. Gangavalli, and A. Thirupathi, "A survey of layer-two blockchain protocols," *Journal of Network and Computer Applications*, vol. 209, p. 103 539, 2023.

[6] L. Bousfield, R. Bousfield, C. Buckland, *et al.*, "Arbitrum nitro: A second-generation optimistic rollup," 2018.

[7] S. Yang, F. Zhang, K. Huang, X. Chen, Y. Yang, and F. Zhu, "Sok: Mev countermeasures: Theory and practice, 2022," *URL: https://arxiv. org/abs/2212.05111*,

[8] P. McCorry, C. Buckland, B. Yee, and D. Song, "Sok: Validating bridges as a scaling solution for blockchains," *Cryptology ePrint Archive*, 2021.

[9] A. Mamageishvili, M. Kelkar, J. C. Schlegel, and E. W. Felten, "Buying time: Latency racing vs. bidding for transaction ordering," in *5th Conference on Advances in Financial Technologies (AFT 2023)*, Schloss-Dagstuhl-Leibniz Zentrum für Informatik, 2023.

[10] Metis, *Decentralized sequencer: Sequencer node*, Accessed: 2024-04-26, 2024. [Online]. Available: https://docs.metis.io/dev/decentralized-sequencer/overview/sequencer-node.

[11] M. Kelkar, S. Deb, S. Long, A. Juels, and S. Kannan, "Themis: Fast, strong order-fairness in byzantine consensus," *Cryptology ePrint Archive*, 2021.

[12] K. Klaus, "Wendy, the good little fairness widget: Achieving order fairness for blockchains," in *ACM Conference on Advances in Financial Technologies*, 2020, pp. 1353–1370.

[13] Y. Zhang, S. Setty, Q. Chen, L. Zhou, and L. Alvisi, "Byzantine ordered consensus without byzantine oligarchy," in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, 2020, pp. 633–649.

[14] Y. Sokolik and O. Rottenstreich, *Age-aware fairness in blockchain transaction ordering. in 2020 ieee/acm 28th international symposium on quality of service (iwqos)*, 2020.

[15] M. Nassar, O. Rottenstreich, and A. Orda, "Communication-aware fairness in blockchain transaction ordering," in *2022 IEEE 23rd International Conference on High Performance Switching and Routing (HPSR)*, IEEE, 2022, pp. 175–182.

[16] H. Zhang, L.-H. Merino, V. Estrada-Galinanes, and B. Ford, "F3b: A low-latency commit-and-reveal architecture to mitigate blockchain front-running," *arXiv preprint arXiv:2205.08529*, 2022.

[17] J. Bebel and D. Ojha, "Ferveo: Threshold decryption for mempool privacy in bft networks," *Cryptology ePrint Archive*, 2022.

[18] D. Yakira, A. Asayag, G. Cohen, *et al.*, "Helix: A fair blockchain consensus protocol resistant to ordering manipulation," *IEEE Transactions on Network and Service Management*, vol. 18, no. 2, pp. 1584–1597, 2021.

[19] A. Chaboud, E. Hjalmarsson, and F. Zikes, "The evolution of price discovery in an electronic market," *Journal of Banking & Finance*, vol. 130, p. 106 171, 2021.

[20] A. J. Hallinan Jr, "A review of the weibull distribution," *Journal of Quality Technology*, vol. 25, no. 2, pp. 85–93, 1993.

[21] P. Virtanen, R. Gommers, T. E. Oliphant, *et al.*, "Scipy 1.0: Fundamental algorithms for scientific computing in python," *Nature methods*, vol. 17, no. 3, pp. 261–272, 2020.

[22] Y. Balasko and H. Crès, "The probability of condorcet cycles and super majority rules," *journal of economic theory*, vol. 75, no. 2, pp. 237–270, 1997.

Authorized licensed use limited to: INDIAN INSTITUTE OF TECHNOLOGY BOMBAY. Downloaded on February 01,2026 at 15:00:47 UTC from IEEE Xplore. Restrictions apply.